Rendre visible l'invisible des communications entre agents grâce à la réalité augmentée

Bleunvenn AUDREN
TELECOM Nancy
Promotion 2027
bleunvenn.audren@telecomnancy.eu

Anne REGENNASS
TELECOM Nancy
Promotion 2027
anne.regennass@gmail.com

Pascale MARANGE
et Rémi PANNEQUIN
Enseignants-Chercheurs au CRAN
Équipe MPSI
pascale.marange@univ-lorraine.fr
remi.pannequin@univ-lorraine.fr

Résumé

Les systèmes multi-agents (SMA) constituent un outil central dans le développement de l'industrie 4.0. Leur fonctionnement est basé sur une transmission d'informations constante entre les différents agents. Ces interactions sont essentielles pour aborder et comprendre le fonctionnement de ces systèmes.
Cependant, elles sont immatérielles et sont donc difficilement intelligibles sans regarder les messages à la source, qui peuvent eux-mêmes être complexes à comprendre.
L'objectif de ce projet est alors de rendre visibles ces échanges d'informations afin de mieux appréhender le comportement des différents agents.

Mots-clés

- ***Réalité augmentée***
- ***Programmation par agent***
- ***Régénération***
- ***Industrie 4.0***

1. Introduction

A. Contexte général

L'industrie 4.0 désigne "une nouvelle manière d'organiser les moyens de production" [1] axée sur l'intégration des nouvelles technologies telles que l'Internet des objets ou l'analyse de données massives (*big data*). Cette révolution industrielle est particulièrement visible à travers le déploiement de chaînes de production intégrant la robotique avancée. Cette évolution émerge dans un contexte marqué par de forts enjeux environnementaux, où l'adoption de ces technologies vise non seulement à accroître la productivité et à réduire les coûts, mais également à optimiser la consommation énergétique.

Ces problématiques éveillent l'intérêt de nombreux chercheurs, notamment au sein du Centre de Recherche en Automatique de Nancy (CRAN), qui étudient les leviers de transition vers une industrie plus durable à travers le concept de régénération. La régénération se définit comme un ensemble de processus visant à redonner un état fonctionnel acceptable à un produit ou à ses composants afin de prolonger leur cycle de vie. Pour soutenir ces travaux, la plateforme ProGreSS4.0 (PROduction & REGeneration SyStem 4.0) a été développée au sein de l'Atelier Inter-établissements de Productique (AIP) Lorrain.

Cette plateforme est constituée d'une ligne de production flexible comprenant plusieurs postes robotisés et d'un magasin automatisé dédié au tri et au stockage des composants selon leur niveau de dégradation. Afin de piloter cet environnement de manière dynamique et adaptative en fonction de l'état de santé de chaque produit, l'architecture logicielle repose sur un Système Multi-Agents (SMA). Ce choix technologique permet d'apporter la modularité et la réactivité nécessaires aux opérations complexes de désassemblage et de diagnostic.

B. Problématique

Bien que la programmation par agents s'impose naturellement pour gérer la flexibilité de la plateforme ProGreSS4.0, elle soulève des difficultés majeures en termes de visibilité et de compréhension de son fonctionnement interne. En effet, lors des démonstrations du système, il s'avère complexe d'expliquer et de rendre intelligibles les échanges de messages et les prises de décisions des agents pour un observateur non initié. Bien que ces interactions soient le cœur du pilotage, leur nature immatérielle rend le fonctionnement global opaque sans une analyse fastidieuse des journaux de communication.

Dans ce contexte, les technologies de réalité augmentée (RA) offrent des perspectives prometteuses pour matérialiser visuellement ces flux d'informations de manière intuitive et pédagogique. Cependant, l'intégration de la réalité augmentée pour superviser un système multi-agents en temps réel soulève plusieurs verrous techniques et scientifiques :

- **La densité de l'information :** Face à la quantité massive de données échangées entre les agents, quelles informations s'avèrent cruciales à afficher pour l'utilisateur ? Comment filtrer l'essentiel pour éviter de surcharger l'environnement visuel de l'opérateur ?
- **La dynamicité temporelle :** Les interactions entre agents sont extrêmement rapides, atteignant parfois plusieurs dizaines de messages par seconde. Pour rendre ces flux perceptibles par l'œil humain, faut-il ralentir artificiellement les communications en mode simulation, ou privilégier un système de relecture en différé (*replay*) ?
- **L'ancrage spatial et la calibration :** Comment garantir que le positionnement des informations virtuelles affichées dans le casque de réalité augmentée reste parfaitement synchrones et cohérentes avec la position physique des robots et des produits ? Des procédures de recalibrage en cours de simulation sont-elles requises, et comment les automatiser ?

L'ensemble de ces défis nous amène à formaliser la problématique centrale suivante : *Comment concevoir et implémenter une interface en réalité augmentée capable de filtrer, d'adapter temporellement et d'ancrer spatialement la visualisation des interactions d'un système multi-agents pour en améliorer la compréhension humaine ?*

2. Revue de Littérature

A. Systèmes multi-agents industriels et problématique de l'opacité

Dans le cadre de l'Industrie 4.0, la recherche de flexibilité pousse les usines à adopter des architectures décentralisées. Les systèmes multi-agents (SMA) s'imposent comme une solution performante pour orchestrer des processus industriels de manière autonome, comme l'illustre la plateforme de recherche et d'enseignement ProGreSS4.0. Dans ces environnements, chaque équipement ou entité logicielle est représenté par un agent autonome (développé par exemple sous le framework JADE) qui collabore avec les autres via des messages standardisés.

Cependant, cette approche distribuée pose un problème majeur d'opacité pour les opérateurs humains. Les décisions et les négociations entre agents se déroulent de manière purement logicielle et asynchrone, ce qui rend le comportement global du système invisible sur le terrain. Sans un outil adapté pour intercepter et matérialiser ces flux d'informations, il devient extrêmement difficile de comprendre la logique interne des agents, de diagnostiquer les pannes ou de former les utilisateurs. C'est pour lever cette barrière cognitive que l'utilisation de la réalité augmentée (RA) devient pertinente, afin de superposer des données virtuelles directement sur les éléments physiques de l'atelier [2].

B. Localisation et tracking spatial par vision par ordinateur

Pour projeter ces flux d'informations visuels de manière cohérente dans l'espace de travail, l'appareil de réalité augmentée doit impérativement être capable de se localiser de façon autonome et ultra-précise. Dans ce domaine, la localisation de caméra basée sur l'image et la cartographie simultanée font l'objet d'études approfondies, les algorithmes de SLAM (*Simultaneous Localization and Mapping*) représentant la brique technologique fondamentale [3]. Le principe du SLAM est de permettre à un appareil de cartographier un environnement inconnu tout en calculant, en temps réel, sa propre position à l'intérieur de cet espace. En analysant les images successives capturées par ses caméras embarquées, le casque évalue ses propres mouvements et met constamment à jour la carte en trois dimensions de la pièce. Parmi les solutions de référence, l'architecture ORB-SLAM s'est distinguée par sa capacité à utiliser des caméras monoculaires standards pour reconstruire l'environnement en temps réel et calculer la trajectoire de l'utilisateur grâce à la détection de points d'intérêt visuels robustes [4].

Néanmoins, le passage de ces modèles théoriques à une application concrète en milieu industriel soulève des défis techniques complexes. Les techniques de localisation de caméra par l'image se heurtent souvent aux variations lumineuses des ateliers, aux textures répétitives ou aux surfaces métalliques qui génèrent des reflets [3]. Ces perturbations altèrent la reconnaissance des points clés géométriques, provoquant des erreurs d'estimation de la pose et un phénomène de dérive (*drift*) [3], [4]. En réalité augmentée, cette dérive désynchronise l'affichage virtuel par rapport aux machines réelles, ce qui justifie la mise en place de points d'ancrage fixes au démarrage de l'application pour stabiliser le repère spatial.

C. Évaluation des dispositifs commerciaux et exigences de tracking

L'évolution récente des technologies immersives a permis l'émergence de casques de réalité mixte grand public et industriels performants, à l'image du MetaQuest 3. Le choix et l'intégration de ces matériels nécessitent d'analyser finement ce que les appareils commerciaux délivrent réellement en matière de tracking spatial et de gestion des ancrages dans un espace de production. Les benchmarks actuels montrent que si la précision des capteurs embarqués est aujourd'hui suffisante pour des interactions en temps réel, les contraintes matérielles d'un atelier (occlusions causées par le passage d'opérateurs, déplacements de robots mobiles) mettent constamment les algorithmes de tracking à l'épreuve [2].

Dès lors, pour concevoir une application de supervision efficace, l'interface ne doit pas seulement afficher des données brutes, elle doit intégrer des principes de transparence et d'explicabilité. En combinant une capture rigoureuse des messages inter-agents et une restitution visuelle stable en réalité augmentée, la solution permet de structurer l'information de manière intelligible pour l'humain [2]. C'est cette alliance entre la robustesse du tracking spatial et la clarté des métaphores visuelles qui offre un support d'aide à la décision et de formation adapté aux contraintes industrielles.

3. Méthodologie

A. Analyse du problème

La plateforme de régénération ProGreSS4.0 repose sur une architecture pilotée par un Système Multi-Agents (SMA) développé sous le framework JADE. Vous trouverez ci-dessous en figure 1 le diagramme de classes décrivant les différentes composantes de cette architecture.

Si cette décentralisation de l'intelligence offre une grande flexibilité opérationnelle pour le pilotage des robots, elle génère ainsi un verrou ergonomique majeur : l'opacité des processus de prise de décision.

En effet, le fonctionnement global de l'atelier repose sur une multitude d'échanges de messages au format FIPA-ACL. Vous trouverez en figure 2 un exemple de ces échanges dans le cas de la vérification de la disponibilité d'un robot mobile. On observe grâce à cette figure la multiplicité des échanges pour une requête qui est d'apparence simple. En effet, on observe déjà cinq messages échangés avant les premiers envois de demandes de disponibilité aux robots mobiles. De plus, les autres messages qui suivent s'y additionnent et sont de plus en plus nombreux, en fonction du nombre de robots mobiles, mais aussi en fonction du contexte : on continue à demander aux robots mobiles s'ils sont disponibles jusqu'à ce que l'un d'eux le soit. Ces échanges s'effectuant en seulement quelques instants, il est difficile de prime abord d'imaginer qu'une telle quantité de messages s'échange, et la nécessité d'avoir accès à une représentation visuelle de ces messages devient claire.

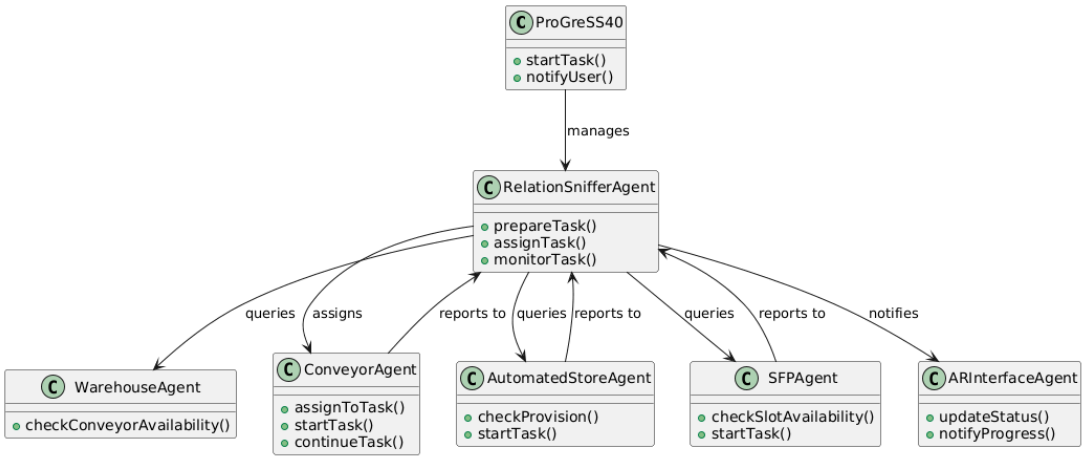


Fig. 1: Diagramme de classes de la plateforme de régénération ProGreSS4.0

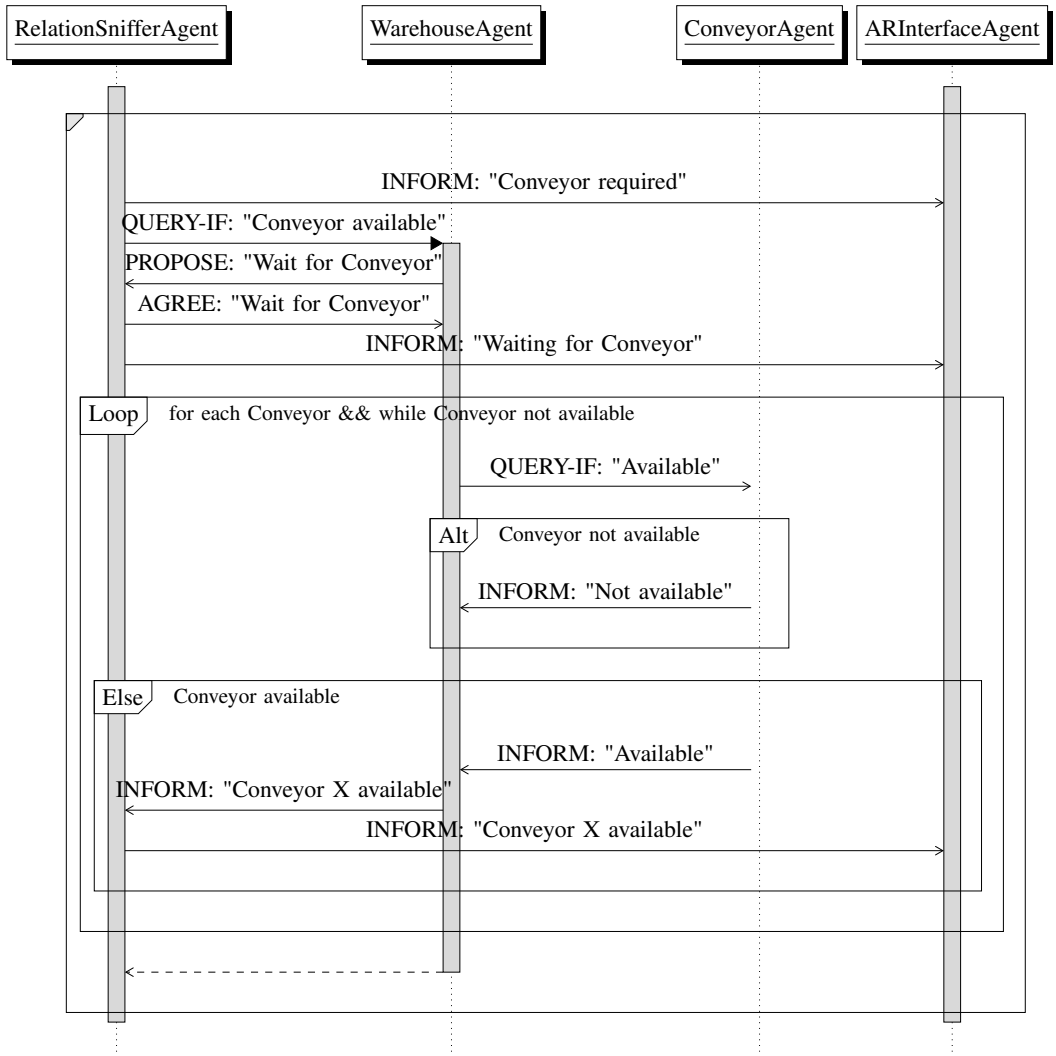


Fig. 2: Diagramme de séquence d'une requête de besoin de robot mobile

L'analyse de cet environnement a mis en évidence trois problématiques fondamentales empêchant la compréhension naturelle de ces flux par un opérateur humain :

- **L'immatérialité et la non-spatialisation :** Les communications logicielles n'ont aucune existence physique. Un message envoyé par l'agent d'un poste de travail à un autre transite sur le réseau sans laisser de trace visuelle dans l'environnement réel de l'AIP Lorrain, rendant le lien de cause à effet invisible.
- **La densité et la vélocité des données :** Les échanges entre agents se produisent à des fréquences informatiques (asynchrones et ultra-rapides) et contiennent des structures de données brutes souvent denses et complexes. Une lecture directe des trames réseau est inadaptée aux capacités d'assimilation d'un utilisateur en temps réel.
- **Le bruit informationnel :** Le système de capture du réseau existant (le *Relation Sniffer*) intercepte la totalité du trafic. Or, une grande majorité de ces messages relève de la synchronisation technique interne (les "battements de cœur" ou *ping* du système) et n'apporte aucune valeur sémantique quant au scénario métier en cours de réalisation (comme le diagnostic ou le tri d'une pièce).

Le cœur du problème d'ingénierie auquel ce projet doit répondre ne se limite donc pas à une simple interception de données. Il s'agit de concevoir un système de supervision capable de créer un pont entre la couche réseau (informatique) et l'espace physique (l'atelier).

Cette analyse a servi de socle à notre démarche, dictant la nécessité d'une abstraction forte des données avant toute implémentation graphique sous Unity.

B. Étude comparative et choix du matériel

Le succès de notre application de visualisation immersive dépend directement des performances et de l'adéquation du matériel retenu. Notre objectif étant de déployer une solution pérenne et fluide au sein de l'AIP Lorrain, une phase d'évaluation comparative a été menée entre les deux équipements mis à notre disposition : le casque *Microsoft HoloLens 1* et le *Meta Quest 3*.

Le tableau en annexe A synthétise les principales caractéristiques techniques et logicielles qui ont guidé notre arbitrage.

1) La compatibilité avec l'écosystème Unity et les contraintes logicielles :

Le facteur le plus critique ayant dicté notre choix réside dans la compatibilité logicielle avec le moteur de jeu 3D Unity, choisi pour développer notre application de supervision.

Le casque HoloLens 1, développé par Microsoft, souffre d'une obsolescence logicielle majeure. Microsoft ayant officiellement abandonné le support de la plateforme *Windows Mixed Reality* (WMR), ce casque ne peut fonctionner avec les versions modernes d'Unity. Son développement impose l'utilisation de versions anciennes d'Unity (antérieures à 2020) et de packages obsolètes[6].

De plus, l'infrastructure de l'HoloLens 1 pose de profonds problèmes de compatibilité. Son écosystème de développement est strictement verrouillé et dépendant de l'environnement Windows. Cela génère des conflits majeurs et une incompatibilité totale avec d'autres systèmes d'exploitation (tels que macOS ou Linux) qui peuvent être utilisés sur les postes de travail de l'équipe ou requis par d'autres briques logicielles du projet multi-agents.

À l'inverse, le Meta Quest 3 offre une compatibilité totale, native et multiplateforme (*cross-platform*). Il supporte pleinement des versions stables d'Unity en support à long terme (*Unity 2022 et 2023 LTS*) via le standard universel *OpenXR*. Les outils de développement et de simulation associés au SDK de Meta sont accessibles aussi bien sur Windows que sur d'autres systèmes d'exploitation, garantissant la flexibilité de notre flux de travail, ainsi que la stabilité et la maintenabilité du code produit[5].

2) Avantages ergonomiques et perception de l'environnement:

Au-delà de la dimension purement logicielle, le Meta Quest 3 se distingue par des capacités matérielles supérieures, indispensables en milieu industriel.

D'une part, son architecture *Video See-Through* (Pass-through couleur) offre un champ de vision étendu de $110^\circ \times 96^\circ$, contre seulement $30^\circ \times 17.5^\circ$ pour l'HoloLens 1. Cette différence est majeure pour l'opérateur : elle évite l'effet de « vision en tunnel » et permet d'afficher des bulles de communication et des flux d'agents larges sans qu'ils ne soient coupés par les bords de l'écran.

D'autre part, la présence d'un capteur de profondeur *Time-of-Flight* (ToF) sur le Meta Quest 3 facilite le scan de la pièce et la gestion de l'occlusion. Les flèches de communication virtuelles peuvent ainsi passer de manière cohérente derrière des obstacles physiques réels (machines, poteaux ou opérateurs), ce qui renforce le réalisme de la simulation.

3) Choix final:

En conclusion, face aux limitations bloquantes de l'HoloLens 1 en matière de support logiciel, de son manque d'ouverture vers d'autres systèmes d'exploitation que Windows, et compte tenu des exigences techniques de notre jumeau numérique sous Unity, le choix s'est logiquement et définitivement porté sur le **Meta Quest 3**.

C. Phase de définition : formulation de la proposition

Cette étape a consisté à définir précisément le périmètre fonctionnel de notre application de réalité mixte. Afin de guider le processus de développement, nous avons formalisé un cahier des charges technique structuré autour de quatre exigences majeures :

- **Visualisation spatiale :** La solution doit permettre de projeter et de superposer les flux de messages informatiques immatériels au sein d'un espace tridimensionnel, en parfaite adéquation géométrique avec l'environnement physique réel de l'AIP Lorrain ou sa réplique simulée.
- **Intuitivité et accessibilité :** Les représentations visuelles retenues doivent limiter la charge cognitive. L'interface doit être immédiatement intelligible, permettant à un opérateur ou à un utilisateur non-expert de saisir le sens général des interactions sans effort de sur-interprétation.
- **Temporalité adaptative :** Le système doit être capable de moduler l'échelle de temps de la simulation. Cette flexibilité est indispensable pour ralentir des échanges asynchrones ultra-rapides (de l'ordre de la microseconde) ou, à l'inverse, condenser des séquences logistiques s'étalant sur de longues minutes.
- **Scénarisation didactique :** L'affichage doit s'appuyer sur des scénarios d'usage clairs et prédéfinis. Ces scénarios agissent comme une trame narrative guidant l'utilisateur pas à pas pour expliciter la dynamique de coopération et le rôle de chaque agent lors des échanges.

D. Conception de la solution et choix de design

Durant la phase de conception, deux décisions architecturales fondamentales ont été prises pour répondre aux exigences graphiques et de positionnement :

- **Interface spatiale par bulles informatives :** Pour matérialiser visuellement l'état interne des agents et le contenu de leurs messages, nous avons choisi d'implémenter des panneaux textuels flottants dynamiques. Positionnés au-dessus des entités physiques (robots, postes), ces éléments textuels sont programmés pour faire constamment face à l'utilisateur, optimisant ainsi la lisibilité.
- **Ancrage spatial par initialisation fixe :** L'alignement précis entre le monde virtuel d'Unity et l'atelier réel constituant un verrou critique, nous avons conçu un protocole basé sur un point de départ d'origine fixe et connu dans l'espace. Lors du lancement de l'application, l'opérateur doit impérativement se positionner à cet emplacement physique précis afin de synchroniser le système de coordonnées du casque Meta Quest 3 avec le repère cartographique de la plateforme.

E. Démarche d'implémentation

Le déploiement technique de la solution au sein du moteur 3D Unity s'est articulé autour de trois jalons opérationnels complémentaires :

1) **Scripting et configuration des scénarios :** Programmation de la suite séquentielle d'événements et des trames de messages à instancier pour rejouer fidèlement le cas d'utilisation industriel sélectionné.
2) **Reconstitution cartographique de l'environnement :** Importation et traitement des données géométriques issues de la carte réelle des agents dans l'éditeur Unity. Cette étape a permis de reconstruire virtuellement les points clés de l'environnement de l'AIP Lorrain pour assurer le placement rigoureux des modèles 3D et la cohérence spatiale des futures interfaces.
3) **Développement du moteur d'affichage des flux :** Programmation sous Unity des scripts gérant l'apparition, les animations cinématiques de déplacement des messages, ainsi que la mise à jour dynamique du contenu textuel des bulles d'information au rythme du scénario.

4. Présentation des résultats

Cette section présente les différentes fonctionnalités obtenues au cours du projet .

A. Cartographie de l'espace de travail

Le premier besoin dans ce projet est de localiser le casque de réalité augmentée dans l'espace. En effet, sans un bon positionnement de celui-ci, l'affichage des messages ne peut être efficace, il peut même devenir incompréhensible ou encombrant. Il faut donc réussir à se positionner dans l'espace de travail pour afficher de manière cohérente les informations . Pour ce faire, nous allons exploiter des données récoltées par certains robots.

En effet, les robots mobiles ont cartographié la zone de la plateforme. Cette carte, sous forme de point, est commune à tous les robots mobiles. Nous l'avons alors récupéré pour recréer cet espace dans Unity. En effet, une fois cette carte recréée, nous pouvons ainsi savoir facilement où sont les différents points d'intérêt de la plateforme (l'emplacement du magasin, les postes des robots fixes, les stations de charge des robots mobiles, les différents objectifs à atteindre). En parsant le document, nous obtenons des zones interdites (*"Forbidden Area"*), des lignes interdites (*"Forbidden Lines"*), ou des objectifs (*"Goal"*) de nos robots mobiles. Les différents points étant définis par leurs coordonnées (données en cm par les robots), on a alors une cartographie précise de la plateforme de l'AIP Lorrain, qui représente une base solide pour, notamment, positionner correctement les sources et destinations des échanges.
Vous trouverez ainsi sur la figure 3 ci-dessous la carte ainsi obtenue vue de haut.

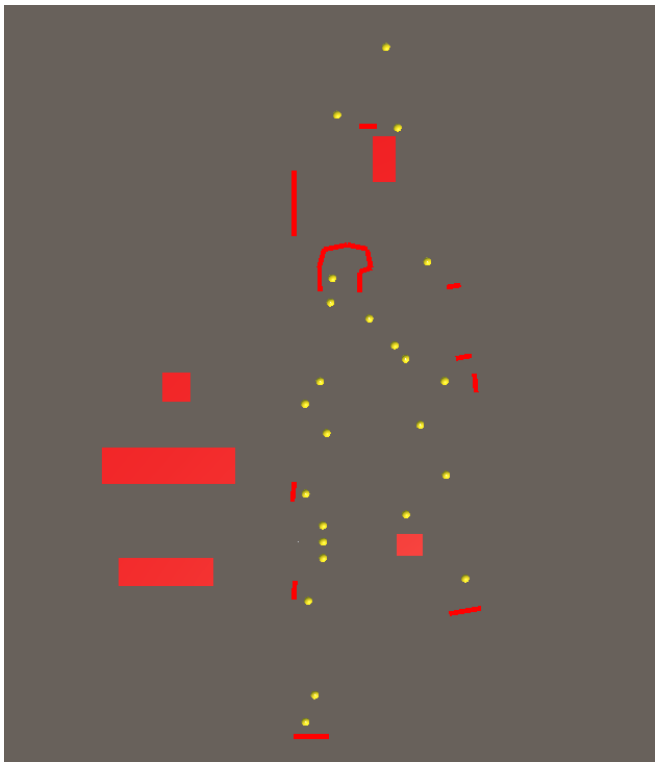


Fig. 3: Cartographie de l'AIP Lorrain vu de haut

Les zones et lignes rouges représentent les zones interdites aux robots mobiles (zones qu'ils ne peuvent traverser ou dont l'accès leur est interdit). Les points jaunes représentent ici les points d'intérêts et objectifs des robots mobiles, comme leurs postes de recharge, ou les postes de chargement.

De plus, les points jaunes ont la particularité d'être nommés. En effet, nous avons récupéré les données de la carte des robots afin de pouvoir déterminer à quoi correspondent les différents points d'intérêt. Vous pouvez ainsi visualiser sur la figure 4 les points dans l'espace correspondant, par exemple, au Magasin Automatique (*MagAuto*).

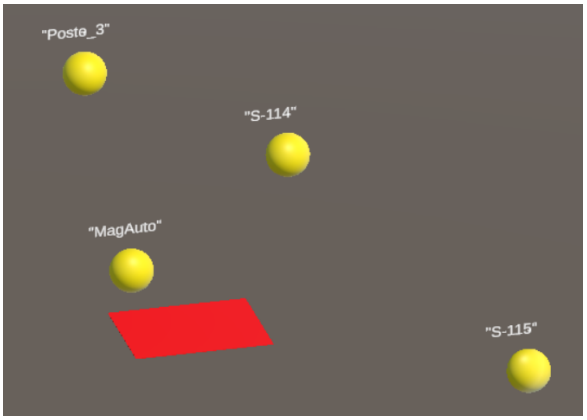


Fig. 4: Visualisation de la nomenclature des points d'intérêts sur la carte générée de l'AIP Lorrain

B. Récupération et traitement des messages

Les fichiers contenant les données des informations échangées sont denses : il faut donc récupérer les différents messages et les trier pour savoir lesquels afficher et comment. Une plus grande compréhension de leur structure est alors nécessaire.

Les données sont composées de plusieurs éléments :

- Un temps *t* qui correspond au moment où le message est envoyé par rapport au début de la séquence.
- Un *type* qui peut être :
  - *BornAgent* : Permet de notifier de la "naissance" d'un agent, de son initialisation.
  - *DeadAgent* : Signale l'arrêt d'un agent
  - *Message* : Permet d'envoyer un message à un autre agent.
  - *TelemetryMessage* : Permet d'enregistrer des données, telles que des coordonnées, des statuts, etc.
- Une valeur *value* qui dépend de son type.
  - *BornAgent* : Composé de l'identifiant de l'agent et de son type.
  - *DeadAgent* : Pas de valeur.
  - *Message* : Composé principalement d'une valeur de *performative* qui donne un entier caractéristique du type de message envoyé, l'agent qui reçoit le message et celui qui l'envoie, le contenu du message, un identifiant de la conversation à laquelle appartient le message, et un identifiant permettant de savoir à quel message celui-ci répond, et quel identifiant doit-on utiliser dans le futur pour indiquer que l'on répond à ce message.
  - *TelemetryMessage* : Composé de l'identifiant de l'agent concerné, le type de l'agent, la date, et soit un statut particulier (caractérisé par une chaîne de caractères), des coordonnées ou une liste de différents objectifs. Le contenu de ce dernier argument dépend de la nature de l'agent qui envoie ce message.

C. Modélisation des échanges

Après avoir traité ces deux parties du projet, il ne nous restait plus qu'à déterminer comment modéliser les transferts de données. Un premier test fut réalisé sur des formes simples pour observer différentes manières de modéliser ces interactions. Les messages sont alors modélisés par des sphères simples, se déplaçant entre deux cubes, symbolisant, d'un côté l'agent qui envoie le message, et de l'autre celui qui le reçoit.
En figures 5 et 6, vous trouverez deux images du même message, représenté par une sphère blanche, allant d'un agent à un autre, de gauche à droite, à deux instants différents.

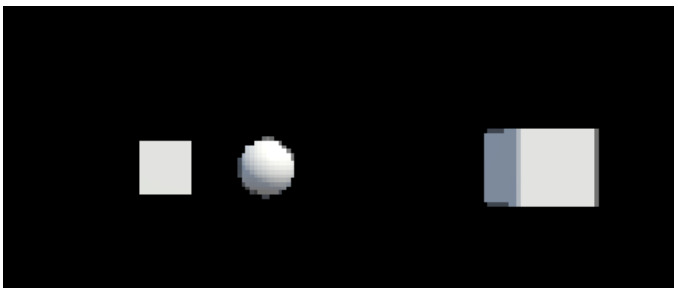


Fig. 5: Test de modélisation de l'envoi d'un message d'un agent à un autre à un instant t

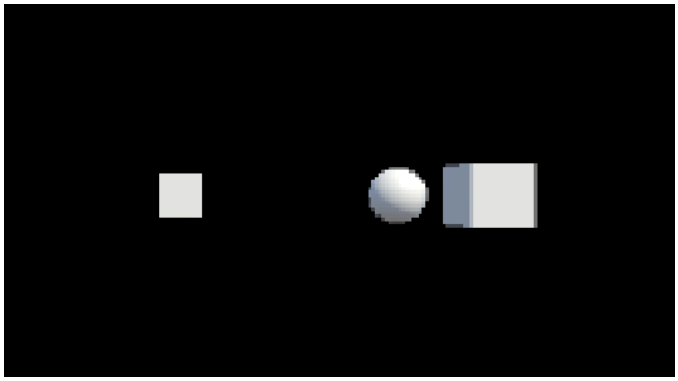


Fig. 6: Test de modélisation de l'envoie d'un message d'unagent à un autre à un instant $t + 1$

D. Fusion et finalisation du projet

Une fois ces différentes parties exécutées, il ne reste donc plus qu'à tout fusionner : les échanges modélisés précédemment étant définis par leur point de départ et d'arrivée, il faut alors adapter ces différentes valeurs en fonction des postes sur la carte et continuer à afficher les partages de données les uns après les autres.

5. Conclusion

A. Bilan et résultats obtenus

En conclusion de ce projet, nous avons réussi à développer un programme permettant de matérialiser les communications multi-agents au sein de la plateforme ProGreSS4.0 via le casque de réalité mixte Meta Quest 3. Pour un scénario d'usage prédéfini, l'application développée sous Unity parvient à superposer l'activité réseau à l'environnement physique de l'AIP Lorrain.

En s'initialisant depuis un point d'origine spatialement défini, le dispositif exploite les données cartographiques fournies pour aligner son repère virtuel avec les installations réelles. Lors de l'exécution du scénario, l'opérateur est en mesure de suivre la chronologie des interactions grâce à des éléments graphiques spatialisés, tels que les bulles de communication. L'objectif principal est ainsi atteint : les échanges d'informations, par nature invisibles et denses, deviennent clairs, visuels et intelligibles pour l'humain.

B. Limites actuelles et pistes d'amélioration

Bien que fonctionnelle, la solution actuelle présente certaines limitations offrant de solides perspectives d'évolution.

La première limite est d'ordre spatial. Pour que la superposition holographique soit exacte, le programme exige que l'utilisateur se place manuellement à un point de départ précis et immobile au lancement de l'application. Sans cette contrainte d'initialisation, l'environnement réel ne sera pas superposé à celui du casque et des décalages pourront être observés lors de l'exécution.

La seconde limite est d'ordre temporel et dynamique. Actuellement, notre programme simule un scénario fixe et scripté, ce qui ne permet pas d'afficher le trafic réseau de la plateforme en temps réel continu.

Pour pallier ces limites, plusieurs améliorations peuvent être envisagées pour les futurs travaux, à différentes échelles :

- **À court terme (Déclenchement événementiel) :** L'application pourrait être modifiée pour réagir à des notifications. Le système resterait en veille jusqu'à ce qu'un agent logiciel prévienne le casque de l'imminence d'une séquence critique (un scénario), déclenchant ainsi l'affichage de manière autonome et ciblée.

- **À long terme (Supervision en temps réel) :** L'objectif final serait de s'affranchir totalement des scénarios pré-enregistrés. En optimisant le traitement des données issues du *Relation Sniffer*, le casque pourrait agir comme un véritable jumeau numérique, capable de filtrer, traiter et afficher les flux d'informations du réseau industriel de manière continue et dynamique.
- **À court terme (Initialisation de la position) :** Un autre objectif, cette fois centré sur la question spatiale, est de permettre à l'utilisateur de lancer l'application de n'importe quel endroit sur la plateforme. Pour ce faire, le casque pourrait détecter des éléments clés de son environnement pour se repérer (tags, qr codes, éléments permanents et caractéristiques du lieu), sans imposer de point de départ à l'utilisateur.
- **À long terme (Synchronisation spatiale en temps réel) :** L'objectif final concernant le positionnement dans l'espace serait de permettre au casque de se synchroniser au cours de la simulation. De manière manuelle ou automatique, le casque pourrait utiliser les mêmes méthodes qu'énoncées précédemment pour se recalibrer et ainsi diminuer, voire supprimer, les décalages ou propagations d'erreurs de position.

REMERCIEMENTS

Nous souhaiterions dans un premier temps adresser notre profonde reconnaissance à P. Marangé et R. Pannequin pour leurs précieuses indications, leur accueil à l'AIPL ainsi que leur encadrement tout au long de ce projet de recherche. Le sujet était très intéressant et nous a été très bénéfique à toutes les deux. Nous remercions également TELECOM Nancy de nous avoir permis de collaborer avec deux chercheurs du CRAN sur ce projet.

REFERENCES

[1] Wikipedia.org - Industrie 4.0
[2] "XR Reality Check: What Commercial Devices Deliver for Spatial Tracking," *arXiv preprint arXiv:2508.08642*, 2025.
[3] "Image Based Camera Localization: an Overview," 2018.
[4] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A versatile and accurate monocular SLAM system," *IEEE Transactions on Robotics*, 2015.
[5] Meta Platforms, Inc., "Site officiel de Meta," https://www.meta.com/, 2026.
[6] Microsoft Corporation, "Documentation et formations Microsoft Learn," https://learn.microsoft.com/, 2026.

ANNEXE A
ÉTUDE COMPARATIVE DES CASQUES DE RÉALITÉ MIXTE OU AUGMENTÉE

Critères	Meta Quest3	Hololens 1
Type de réalité	Passthrough vidéo : retransmission du monde réel par capture vidéo	Optique see-through : vision directe du monde réel
Version compatible Unity	Version 2022 : 2022.3	Version 2017 : 5.6
Champ de vision utile	$110^\circ \times 96^\circ$	$30^\circ \times 17,5^\circ$
Pérennité	Ecosystème actuel	Obsolescence logicielle